

The Skill Is Not the Workflow: Behavioral Patch Review for Reusable LLM Agents

Anonymous Author(s)

Abstract

People increasingly become responsible for recurring work performed by large language model (LLM) agents without directly programming or observing the workflows those agents enact. Their intent is mediated through reusable natural-language artifacts, whether they author those artifacts or review AI-generated revisions. We use agent skills as a concrete entry point into this form of mediated control. A skill is editable, but it is not the workflow: an agent interprets it with task context, surrounding instructions, tools, and model variation. During repair, this distinction matters. A failed execution identifies an unacceptable outcome, not the policy that should govern future cases; a textual patch shows what changed in the artifact, not which future behaviors changed with it. We introduce *behavioral patch review*, an interaction model connecting user intent, skill artifacts, and enacted workflows. `SYSTEMNAME` helps workflow owners construct a repair contract over contrasting contexts, marking behavior that should change, remain stable, or remain unresolved. It uses repeated skill interventions to expose behavioral sensitivity, coupling, and uncertainty, supports AI-assisted patching, and compares intended with observed behavioral change before release. We evaluate the approach through a formative study of real repair episodes, a technical evaluation of intervention validity, and a controlled study of workflow prediction and repair decisions. This work reframes skill repair from editing instructions to governing behavioral change across user intent, skill artifacts, and enacted workflows.

CCS Concepts

• **Human-centered computing** → **Human computer interaction (HCI)**; *Interaction design*.

Keywords

agent skills, LLM agents, interactive debugging, counterfactual evidence, end-user programming, human-AI interaction

Draft integrity note. This working draft contains a placeholder system name, missing-data markers, and provisional findings prose. Replace every placeholder and audit all findings, quotations, analyses, and citations against collected evidence before submission.

1 Introduction

People increasingly own agent workflows that they do not directly program or observe. A domain expert may decide what an agent should prioritize, approve its deployment, and bear the consequences of its actions, while an AI assistant or developer writes the instructions that shape those actions. We call the person with

authority over the intended behavior and its release a *workflow owner*, regardless of whether that person authored the underlying instructions. The owner's control is often mediated through reusable natural-language artifacts such as system prompts, custom instructions, templates, routines, and agent skills. We use *skill* as an umbrella term for such an artifact when it organizes an agent's decisions and actions across a recurring family of tasks. Skills therefore provide a concrete entry point for studying how people understand and govern agent workflows through an editable intermediary.

The skill, however, is not the workflow. It conditions a family of context-specific workflows that emerge when an agent interprets the artifact together with the current task, surrounding instructions, available tools, intermediate results, and model variation. Many workflow owners may never inspect the skill text directly. Even experienced maintainers who understand every sentence need not be able to infer its runtime semantics from text alone. We use *enacted workflow* to denote the externally observable sequence of decisions, tool calls, confirmation points, actions, and outcomes produced in a particular context, rather than the model's hidden reasoning. Skill-mediated control thus links three distinct objects: the behavior the owner intends, the artifact that encodes or approximates that intent, and the workflow the agent enacts.

Consider a researcher who relies on a travel-recovery skill. The skill checks affected commitments, retrieves feasible alternatives, compares arrival time and cost, applies airline and connection preferences, requests confirmation above a fare threshold, and then rebooks. After a cancellation, the agent selects an \$80 same-airline flight arriving the following afternoon over a \$420 flight arriving that evening. The choice satisfies cost and airline preferences but causes the researcher to miss a paper presentation the next morning. The researcher can judge that this outcome is unacceptable. What remains unclear is how the recurring workflow should change. The calendar check may have failed. The cost rule may be too strong, the airline preference too broad, or the skill may lack a priority rule that distinguishes fixed commitments from routine travel.

Several edits can repair the visible incident while encoding different future policies. One patch could always prioritize the earliest arrival. Another could prioritize arrival only when a fixed commitment is at risk. A third could relax cost and airline preferences for time-sensitive travel while preserving confirmation for unusually expensive purchases. All three may produce the desired flight in the motivating case, yet they behave differently on routine trips, flexible meetings, expensive alternatives, and cases in which airline or connection preferences should still dominate. The failure therefore identifies an objection, not a specification. It exposes an *articulation gap*: the owner can reject an outcome before being able to state the reusable policy that should replace it. Repair requires a relative specification of behavioral change, including what should change, what should remain stable, and which trade-offs remain unresolved.

A chosen patch introduces a second problem. Natural-language rules can overlap, override one another, activate only in particular contexts, or produce variable executions. A local textual edit can therefore have a nonlocal behavioral effect. The textual diff reveals which words changed in the skill; it does not reveal which decisions, tool calls, or outcomes changed in the enacted workflow. This creates an *enactment gap* between the editable artifact and its runtime behavior. Because owners observe that behavior through only a small set of executions, it also creates a *legibility gap* between the enacted workflow and the owner's expectations of future behavior. A successful rerun of the motivating case does not establish that the repair transfers, preserves previously useful behavior, or remains stable enough for reuse.

Existing systems support important parts of this process. Prompt-authoring tools compare configurations and evaluate outputs against explicit criteria [1, 8]. Interactive debugging and what-if systems expose traces or alternative execution branches [4, 14]. Policy-authoring systems help practitioners articulate behavioral regions and grow test sets around ambiguous rules [11, 12]. Automated methods revise skills from execution evidence and select candidates using task verifiers [13]. Taken together, these systems make two edges of skill-mediated control increasingly inspectable: they help people translate intent into prompts or policies, and they help developers compare artifacts or executions. A distinct problem remains in closing the loop across all three layers when no single verifier captures the owner's domain-specific trade-offs. The owner must connect an evolving statement of intended change to the skill artifact, inspect how that artifact shapes behavior across contexts, and decide whether the observed change is aligned and stable enough for release.

We ask: *How can interactive systems help workflow owners understand and govern agent workflows that they control indirectly through reusable natural-language skills?* We study this question through repair episodes, because a breakdown makes the relationship among intent, artifact, and enacted behavior consequential and contestable. Our focus is not to reconstruct hidden reasoning or diagnose every component of an agent system. It is to make the behavioral consequences of changing a persistent control artifact reviewable. We frame this activity as *behavioral patch review*. As in software patch review, a proposed change must be inspected before release. Unlike deterministic code, however, the relevant behavioral diff must be estimated from repeated executions, and the intended diff may itself emerge through the owner's judgments over concrete cases. The goal is not merely a locally successful textual patch. It is a change that affects no more and no less of the workflow than the available evidence and the owner's intent justify. A good repair is not necessarily the smallest textual edit; it is the smallest justified behavioral change.

We instantiate behavioral patch review in `SYSTEMNAME`. First, the system helps the owner construct a progressively refined *repair contract*. Across the motivating failure and contrastive contexts, the owner marks behavior that should *change*, *remain stable*, or remain *unresolved*. The contract evolves as new cases expose tacit requirements or policy boundaries. These case-level judgments, rather than manual instruction editing, are the owner's primary unit of control. Second, `SYSTEMNAME` runs repeated, controlled

interventions on individual skill instructions and selected combinations. It treats these interventions as probes rather than definitive root-cause diagnoses, using them to expose behavioral sensitivity, coupling among instructions and requirements, evidence stability, and inconclusive cases. Third, the system can propose candidate textual patches while the owner retains authority over behavioral intent and release. Finally, it executes the original and revised skills across reviewed contexts and compares the contract's *intended behavioral delta* with the *observed behavioral delta* in the enacted workflows. The review surfaces underreach, unintended change, unresolved exposure, and insufficient evidence before the owner chooses to release, revise, gather more evidence, abstain from a skill edit, or escalate the failure beyond the skill layer.

We investigate this interaction model at three levels. A formative study of real repair episodes examines how workflow owners relate intended behavior, reusable instruction artifacts, and enacted workflows when outcomes violate expectations. A technical evaluation tests whether controlled interventions provide stable and decision-relevant evidence, including whether intervention effects correspond to the effects of actual edits. A controlled study evaluates whether behavioral patch review improves calibrated predictions of held-out workflow behavior, reduces mismatch between intended and observed repair scope, and supports better-founded release, revision, evidence-gathering, and abstention decisions. These evaluations treat workflow understanding as an empirical outcome rather than assuming that more explanation or more executions necessarily produce a more accurate mental model.

This work makes three contributions:

- (1) **An empirical account of skill-mediated agent workflow control.** Using repair episodes as a breakdown lens, we characterize how workflow owners reconcile intended behavior, reusable natural-language skill artifacts, and context-specific enacted workflows. We identify articulation, enactment, and legibility gaps that arise when people are responsible for recurring agent behavior without directly programming or observing the workflow.
- (2) **Behavioral patch review, an interaction model connecting user intent, skill artifacts, and enacted workflows.** The workflow supports progressive specification of intended behavioral change, execution-grounded inspection of skill-to-workflow relationships, AI-assisted textual revision, and comparison of intended and observed behavioral deltas before reuse.
- (3) **Technical and human evidence on workflow legibility and repair governance.** We evaluate intervention validity, calibrated predictions of held-out workflow behavior, alignment between intended and observed repair scope, and decisions to release, revise, gather evidence, or abstain.

By treating skills as editable intermediaries rather than transparent workflow specifications, this work shifts the unit of repair from text alone to the relationship among intent, artifact, and enacted behavior. More broadly, it argues that interfaces for maintaining reusable agents should make behavioral change reviewable by the people who own its consequences, even when those people do not directly engineer the workflow.

2 Related Work

Our work draws on end-user programming of AI behavior, interactive control of agent workflows, and testing and maintenance of LLM-based systems. Together, these areas explain how people express intended behavior through reusable natural-language artifacts, inspect the workflows those artifacts induce, and revise them after breakdowns.

2.1 End-User Programming of AI Behavior

End-user programming enables domain experts to shape computational behavior through task-level representations without mastering conventional programming [9]. Reusable natural-language instructions extend this premise to LLMs: they are inspectable and editable, yet their operational effects depend on model interpretation, surrounding context, available tools, and sampling. Prompt-authoring systems such as ChainForge and EvalLM therefore supplement text editing with comparative generation and user-defined evaluation criteria [1, 8]. Policy-authoring systems broaden the unit of design from individual outputs to regions of behavior. Policy Maps supports exploration of an open-ended behavior space, while Data-Prompt Co-Evolution couples prompt revision with edge cases, rationales, neighboring examples, and a living test set [11, 12]. Together, these systems show that examples can help users articulate behavioral boundaries rather than only score a policy fixed in advance. This framing is particularly relevant when the person accountable for workflow behavior has domain authority but does not maintain every underlying instruction. More generally, making a natural-language artifact readable does not make the workflow it induces deterministically predictable.

We build on this view of natural-language artifacts as sites of behavioral specification, but study their maintenance after reuse. In our setting, an artifact has already produced behavior its owner values, and one breakdown motivates a *relative* change. The owner must determine not only what should improve, but what should remain stable and whether the mismatch should be repaired in the user-facing skill at all. Additional cases can surface tacit commitments or unresolved trade-offs, thereby revising the repair specification itself rather than merely evaluating a fixed prompt. Skill maintenance thus connects domain judgment to an editable artifact whose text remains an indirect account of the workflow it will produce.

2.2 Interactive Control of Agent Workflows

Interactive agent systems make runtime activity more actionable through shared representations and counterfactual intervention. Cocoa uses an interactive plan to support co-planning, delegation, execution, and replanning between a user and an agent [5]. Agent-debugging systems expose complementary views of execution: AGDebugger supports navigation, editing, and reset of multi-agent conversation branches, while RAG Without the Lag enables rapid what-if analysis over retrieval and generation components [4, 14]. Together, these systems show that representations connected to runtime actions and alternatives can make agent activity more inspectable and steerable than an undifferentiated chat history. They also establish a useful interaction pattern: users can

intervene in a concrete execution without first reconstructing the agent's full internal process.

These techniques primarily support coordinating, understanding, or steering a current task. A trace or branch comparison provides evidence about one enacted workflow, but not whether an observed effect is stable across repeated runs, transfers across contexts, or falls within the intended scope of a persistent repair. Research on appropriate reliance similarly cautions that fluent explanations can increase reliance on both correct and incorrect outputs, whereas sources and visible inconsistencies can help users detect uncertainty [7]. Evidence for workflow repair should therefore remain traceable to concrete executions and expose disagreement or variation rather than replace them with a plausible root-cause narrative. We extend interactive control from steering one trajectory to reviewing how a revision to a reusable artifact may alter a family of future workflows.

2.3 Testing and Maintenance of LLM-Based Systems

Software maintenance distinguishes repairing a motivating failure from protecting behavior that previously worked. Regression testing checks prior behavior after a change, while code review makes a proposed patch and its rationale inspectable before integration [2, 15]. These practices become less direct for LLM artifacts: executions are probabilistic and context-dependent, and textual locality does not guarantee behavioral locality. PromptPex extracts input-output specifications from a prompt and generates tests that can expose invalid behavior and later regressions [16]. Contrastive Reflection combines failures with nearby successes to propose targeted edits and select candidates using held-out validation and optional regression constraints [10]. These methods establish that prompt repair can be test-driven and regression-aware rather than ending after a satisfactory rerun. PromptPex is most decisive when the prompt already states the relevant contract, whereas Contrastive Reflection assumes that held-out examples or task-level criteria can rank candidate edits. When the artifact is itself incomplete, conflicting, or overly broad, additional evidence may revise what the owner considers the repair contract rather than only score a candidate against a fixed one.

Recent systems extend execution-grounded repair from prompts to persistent skills and agent harnesses. SkillRevise uses execution traces, candidate revisions, re-execution, and empirical utility to improve an initial skill; SkillHEX turns falsifiable failure hypotheses into executable tests and searches persistent revision branches [6, 13]. HarnessFix localizes failed trajectories across layers of an agent harness, applies scoped repair operators, and validates patches on held-out tasks and regressions [3]. These systems demonstrate that execution-grounded diagnosis, revision search, and regression validation can be automated, and they underscore that a visible failure need not originate in the user-facing skill. We build on these mechanisms but examine a different interaction condition: empirical utility, executable tests, or held-out tasks can compare revisions once the relevant target is sufficiently specified, whereas a workflow owner may still be determining the intended policy and whether the skill is the appropriate repair locus.

Our setting therefore differs in what remains unsettled when repair begins. A workflow owner may know that one outcome is

unacceptable while still deciding the policy boundary for neighboring cases, the system layer that should change, and what evidence would justify reuse. Additional executions may change the specification itself, reveal nonlocal consequences of a textual patch, or show that no skill edit is warranted; no single verifier may capture these domain trade-offs in advance. We study the interaction problem of owner-governed maintenance, complementing automatic repair with support for forming a relative behavioral specification, connecting a concrete revision to cross-context evidence, and determining whether a persistent change is justified.

3 Formative Study: Locating and Scoping Repair in Skill-Mediated Workflows

Repair episodes reveal a structural asymmetry in recurring agent use. A workflow owner can recognize that an execution is unacceptable while remaining uncertain whether the mismatch lies in the policy they intended, the reusable artifact that was meant to express it, the workflow enacted in the current context, or another system component. We use *workflow owner* to describe a person who defines the goals of a recurring task, judges the acceptability of agent behavior, or bears responsibility for its consequences, whether or not they authored the underlying skill. We conducted an artifact-grounded formative study to understand how workflow owners locate a breakdown, construct the scope of a possible repair, and distribute repair work between themselves and AI. We asked:

- **FQ1:** How do workflow owners make sense of breakdowns in recurring agent workflows, and what do they treat as possible objects of repair?
- **FQ2:** How do additional cases and execution evidence reshape what workflow owners believe should change, remain stable, or remain unresolved?
- **FQ3:** How do workflow owners distribute responsibility between themselves and AI across evidence gathering, repair implementation, and release?

3.1 Participants, Procedure, and Analysis

We recruited *[FINAL N]* workflow owners who had used an LLM assistant or agent for the same recurring task at least five times and had encountered an execution they considered unacceptable within the previous six months. Eligibility depended on responsibility for the workflow's goals or outcomes, not on authorship of its reusable instructions. We used maximum-variation sampling across participants' familiarity with inspecting or modifying reusable instructions and the consequences of incorrect workflow behavior. Some participants primarily evaluated domain behavior and relied on a teammate or AI to change the configuration; others regularly maintained system prompts, templates, routines, or skills. We treated this variation as a source of qualitative contrast rather than as experimental conditions. Before the session, participants provided a redacted failure execution, a typical or successful execution from the same workflow, and any available instruction, configuration, or revision artifact. Sessions lasted approximately 80 minutes; participants received *[COMPENSATION]*, and the protocol was *[ETHICS/IRB STATUS]*. Appendix E reports recruitment and sample details.

Each session comprised three activities. First, during an *artifact-grounded breakdown reconstruction* (25 minutes), participants used their own materials to reconstruct what happened, what they expected, which evidence they directly observed, what they inferred, and what they changed or considered changing. The interviewer used a neutral event timeline and did not introduce an intent-skill-workflow model or presume that the reusable instructions were at fault. Second, participants completed an *open-ended Wizard-of-Oz repair probe* (35 minutes) built around a common travel-recovery incident. We first showed only the request, observable execution, and unacceptable outcome, and then revealed the reusable skill and available tools. Participants could request whatever evidence they considered useful, including another context, repeated executions, a rule-level what-if, tool-state evidence, or a comparison of candidate revisions. A facilitator retrieved responses from a fixed, precomputed execution bank rather than improvising evidence. After this open request period, every participant received a counterbalanced common evidence set containing a preservation case, a difficult boundary case, repeated runs, and a signal that the failure might not be attributable to the skill. Participants then chose whether to write a revision, ask AI to draft one, describe the desired behavior without editing text, or defer the edit, followed by a release, revision, further-evidence, abstention, or escalation decision. Third, a *reflection and delegation interview* (15 minutes) compared the probe with participants' own practice and examined preferred representations, implementation support, and decision authority. The probe was intentionally lighter-weight than our final system and did not expose its representations or interaction techniques. Appendices G–J provide the complete materials.

We collected submitted artifacts, transcripts, event timelines, evidence requests, successive explanations and repair proposals, behavioral predictions, implementation choices, and final decisions. Analysis proceeded in two stages. We first used inductive event coding to characterize observed symptoms, expected behavior, candidate explanations, evidence sources, uncertainty, proposed actions, scope revisions, and decision rationales without assigning participants' accounts to a predetermined three-layer model. For FQ2, we constructed within-participant process traces linking evidence requested or received to changes in interpretation, repair scope, confidence, and action. For FQ3, we compared participants' enacted choices in the probe with the responsibilities they described in interviews. Only after this first cycle did we conduct an abductive cross-case synthesis to examine whether recurring objects of repair could be understood as intended behavior, reusable artifacts, enacted workflow behavior, or conditions beyond the skill. Two researchers independently examined an initial maximally varied subset, iteratively refined the coding and event schema, and searched the full corpus for disconfirming cases. We analyzed real incidents and standardized-probe behavior separately before comparing them. Descriptive counts characterize patterns within our sample and do not support statistical generalization. Appendix K details the analysis and claim-audit process.

3.2 Findings

3.2.1 F1. Breakdowns did not identify a single object of repair. Across participants' real incidents, *[X of N / NUMBER OF INCIDENTS]* workflow owners could readily explain why an observed outcome was unacceptable, yet lacked a stable account of what should change. Their candidate objects of repair included an under-specified policy, one instruction, an interaction among otherwise reasonable instructions, unstable execution, tool or retrieval state, and the way task context had been presented. Participants frequently moved among these explanations rather than progressing directly from a failed output to a faulty line of text. Importantly, their accounts varied in evidential status: traces exposed some actions or tool results, whereas claims about rule priority or model interpretation were often retrospective inferences. *[REPORT HOW OFTEN PARTICIPANTS REVISED THE PRESUMED OBJECT OF REPAIR DURING THE REAL-INCIDENT RECONSTRUCTION.]*

Access to the reusable artifact changed, but did not necessarily resolve, this uncertainty. Participants with less implementation experience tended to reason from desired outcomes, examples, and operational consequences; experienced maintainers more readily named instructions and interactions. However, *[REPORT WHICH DIFFICULTIES PERSISTED AMONG EXPERIENCED MAINTAINERS, SUCH AS PRIORITY, TOOL EFFECTS, OR STOCHASTICITY]*. The standardized probe provided a common illustration: after seeing the same incident and later the same skill, participants *[REPORT THE RANGE OF OBJECTS OF REPAIR AND HOW MANY CHANGED THEIR EXPLANATION]*. A representative participant distinguished behavioral judgment from implementation certainty: *"[VERIFIED QUOTE ABOUT KNOWING THE BEHAVIOR WAS WRONG BUT NOT KNOWING WHETHER POLICY, SKILL, EXECUTION, OR ANOTHER COMPONENT SHOULD CHANGE]."*

Negative cases bounded this pattern. *[REPORT INCIDENTS IN WHICH THE PARTICIPANT RAPIDLY LOCALIZED THE PROBLEM, AND CASES IN WHICH THEY CONCLUDED THAT A SKILL EDIT WAS UNWARRANTED]*. Taken together, the accounts suggest that workflow accountability and artifact expertise were not equivalent. A skill provided an editable control surface, but reading it did not always provide a stable account of the context-specific workflow the agent would enact. Repair therefore began by locating the possible object of change, not merely by rewriting an identified instruction.

3.2.2 F2. Repair scope was constructed comparatively across cases. Participants initially expressed repair intent in several forms: an objection to the visible outcome, a preferred decision for the current case, a candidate textual edit, or a broader policy. *[REPORT THE DISTRIBUTION OF THESE INITIAL FORMS AND HOW OFTEN INITIAL PROPOSALS OMITTED AN APPLICABILITY BOUNDARY, PRESERVATION COMMITMENT, OR UNRESOLVED TRADE-OFF]*. These first proposals often answered "what should have happened here" more clearly than "which future cases should follow the same rule." The real incidents showed the practical consequence of this gap: *[REPORT HOW MANY PARTICIPANTS DESCRIBED A PREVIOUS LOCAL FIX THAT LATER CHANGED ANOTHER DESIRED BEHAVIOR, DELAYED RELEASE, OR LED TO ROLLBACK/WORKAROUND]*.

The open evidence-request period let us distinguish what participants anticipated from what the study subsequently introduced. Participants spontaneously requested *[REPORT REQUEST TYPES*

AND FREQUENCIES: CONTRASTING CONTEXTS, REPEATED RUNS, TOOL STATE, WHAT-IFS, PATCH COMPARISONS, RAW TRACES]. These requests exposed different uncertainties: some participants sought another case to discover the boundary of a rule, while others wanted repetition to determine whether the observed behavior was stable. The common evidence set then provided comparable pressure on every proposal. A routine case could reveal an established behavior that a broad repair would disturb; a boundary case could surface a trade-off for which the participant had not yet formed a policy; repeated runs could weaken confidence in a single execution; and tool-state evidence could redirect repair away from the skill.

Our process traces captured whether evidence changed more than confidence. After self-requested and common evidence, *[X PARTICIPANTS NARROWED, Y BROADENED, Z REFRAMED, W DEFERRED, AND V ABANDONED THEIR INITIAL REPAIR]*. Preservation commitments first appeared *[WHEN/AFTER WHICH EVIDENCE]*, and unresolved regions appeared *[WHEN/AFTER WHICH EVIDENCE]*. Participants sometimes revised their interpretation of the original workflow at the same time as the proposed repair. As one participant explained, *"[VERIFIED QUOTE SHOWING THAT ANOTHER CASE CHANGED WHAT THE RULE SHOULD MEAN, NOT ONLY CONFIDENCE IN AN EXISTING EDIT]."* We also observed *[NEGATIVE CASES IN WHICH ADDITIONAL EVIDENCE ONLY CONFIRMED A PREVIOUSLY ARTICULATED POLICY]*.

These transitions indicate that contrasting execution was not only a test of a fixed specification. Participants used cases to construct the relative scope of change: where behavior should differ, where it should remain stable, and where the desired policy was not yet settled. Determining that the available evidence did not warrant a skill edit was part of this scoping process rather than a failure to complete repair.

3.2.3 F3. Participants separated evidence and implementation work from policy and release authority. Participants distributed repair work across three forms of responsibility. *Evidence work* included generating cases, repeating executions, conducting what-if comparisons, and organizing results. *Implementation work* included drafting or applying textual revisions. *Policy and release work* included judging acceptable behavior, resolving trade-offs, deciding whether evidence was sufficient, and authorizing reuse. In the probe, participants actually delegated *[REPORT EVIDENCE TASKS AND COUNTS]* and selected *[COUNTS FOR MANUAL EDITING, AI DRAFTING, BEHAVIOR-ONLY SPECIFICATION, DEFERRAL, OR NO EDIT]*. These enacted choices did not always match their initial statements about preferred control: *[REPORT ANY SYSTEMATIC MISMATCH BETWEEN WHAT PARTICIPANTS SAID AND WHAT THEY DID]*.

Artifact familiarity shaped the level at which participants wanted to engage, but it did not map cleanly onto a single allocation of authority. *[REPORT HOW LESS-EXPERIENCED WORKFLOW OWNERS AND EXPERIENCED MAINTAINERS DIFFERED IN THEIR DESIRE FOR TEXTUAL DETAIL, RAW EXECUTIONS, OR MANUAL EDITING]*. Perceived consequence also mattered: *[REPORT WHEN HIGHER CONSEQUENCE PRODUCED MORE CONSERVATIVE DELEGATION OR STRONGER TRACEABILITY DEMANDS]*. At the end of the probe, participants chose *[COUNTS FOR RELEASE, REVISE,*

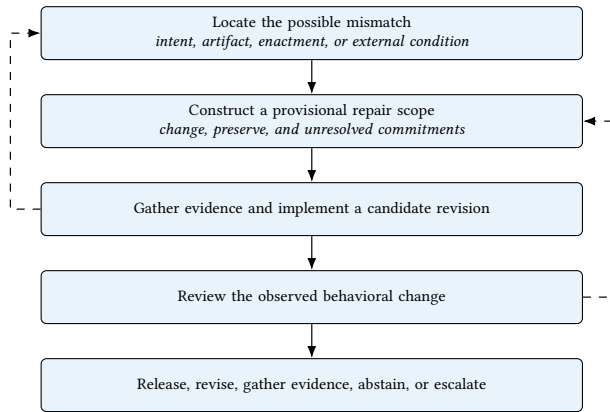


Figure 1: Provisional process model to retain only if supported by the final analysis. Evidence can revise both the presumed location of a breakdown and the intended scope of repair.

MORE EVIDENCE, ABSTAIN, AND ESCALATE] and justified these choices in terms of *[DOMINANT DECISION CRITERIA]*. A participant located their desired authority precisely: “[*VERIFIED QUOTE DISTINGUISHING WILLINGNESS TO DELEGATE EXPERIMENTS OR PATCH DRAFTING FROM AUTHORITY OVER BEHAVIOR OR RELEASE*].”

The accounts were heterogeneous. *[REPORT PARTICIPANTS WHO INSISTED ON DIRECT TEXTUAL AUTHORSHIP, THOSE WILLING TO DELEGATE MOST IMPLEMENTATION, AND ANY WILLING TO DELEGATE POLICY OR RELEASE]*. Rather than equating agency with personally writing every instruction, participants negotiated control across behavioral commitments, textual implementation, access to evidence, and the release decision. This division of labor motivates a workflow in which AI can perform high-effort evidence and implementation tasks while users retain an inspectable path to the underlying artifact and authority over whether a concrete behavioral change is ready for reuse.

Synthesis: repair as cross-layer reconciliation. Together, the findings characterize repair as an iterative process rather than a feature checklist (Figure 1). Workflow owners first locate what kind of mismatch may have occurred, construct a provisional scope of behavioral change across cases, gather evidence and implement a candidate revision, and decide whether the observed change warrants release, further revision, more evidence, or no skill edit. Evidence can send the user backward by changing either the presumed object of repair or the policy boundary itself. This account uses skills as a concrete, editable intermediary between user intent and enacted workflow behavior while preserving the possibility that a breakdown resides outside the skill.

3.3 Design Goals for Behavioral Patch Review

We synthesized the findings into three goals for supporting repair without requiring workflow owners to begin with a correct diagnosis or a complete specification.

DG1: Support cross-layer localization without presuming a skill fault. The interface should begin with an observable breakdown and help users inspect behavioral expectations, reusable instructions, execution evidence, and relevant external conditions. It should preserve multiple plausible objects of repair, distinguish observation from inference, and avoid converting uncertain evidence into a responsible-instruction label. “No skill edit” and escalation to another system layer should remain first-class outcomes [F1, F2].

DG2: Externalize repair scope as revisable behavioral commitments. The interface should help workflow owners progressively articulate where a candidate repair should change behavior, where established behavior should remain stable, and where the desired policy remains unresolved. These commitments should be grounded in concrete contexts and remain editable as additional executions alter the user’s interpretation. The representation should serve both as an evolving specification of intended change and as the reference against which a candidate implementation is reviewed [F1, F2].

DG3: Connect candidate implementations to behavioral evidence while preserving release authority. AI may generate cases, execute probes, organize repeated comparisons, and draft textual revisions. Workflow owners should be able to inspect how a concrete revision changes observable decisions, tool actions, and requirement outcomes, with access to the cases and executions underlying each summary. The workflow should culminate in a choice to release, revise, gather more evidence, abstain, or escalate, allowing users to delegate implementation without surrendering behavioral policy or release authority [F2, F3].

Together, these goals motivate *behavioral patch review*: a shared repair workflow that connects user judgments, reusable skill artifacts, and enacted workflow behavior while making uncertainty and non-skill failures actionable. The next section describes how SYSTEMNAME operationalizes this process.

4 SYSTEMNAME: Reviewing Behavioral Patches to Reusable Agent Skills

Guided by the formative study, we designed SYSTEMNAME as a behavior-first workspace for repairing reusable agent skills without requiring workflow owners to begin from instruction-level diagnosis. The system is organized around one shared representation, the *Behavioral Patch Table* (Figure 2). Each row connects a concrete use case, the workflow’s current observable behavior, the user’s intended relative change, and the behavior enacted by a candidate skill. Rather than exposing these as four equally weighted columns, the interface presents a compact before–after behavioral difference and automatically identifies whether the candidate changed, preserved, or entered a region of behavior as intended. Skill text, individual executions, evaluator details, and controlled what-if tests are available on demand.

A repair proceeds through one iterative loop: the workflow owner (1) scopes a patch with a small set of concrete cases, (2) approves the behavioral intent of an AI- or user-authored skill revision, and (3) reviews mismatches between intended and observed behavioral change. A mismatch can prompt the user to revise the patch, add a case, or invoke a targeted execution probe from that

row. The user ultimately releases the revision, gathers more evidence, or abstains from a skill edit. This design operationalizes the paper's three claims: the skill is not the workflow, a failure is not a specification, and a textual patch is not a behavioral diff.

4.1 User Scenario: Scoping and Reviewing a Behavioral Patch

We illustrate the interaction through Lin, the researcher from Section 1 who relies on a reusable travel-recovery skill. Lin can judge whether a travel decision respects her commitments, costs, and confirmation preferences, but she does not know how the skill's instructions combine at runtime. The scenario demonstrates the system workflow; it is not presented as evaluation evidence.

4.1.1 From a breakdown to a small repair scope. Lin opens the execution in which the agent selected a cheap flight that arrived after her conference presentation. SYSTEMNAME imports the task, the versioned skill, and the observable execution trace, and creates the first row in the Behavioral Patch Table. Instead of asking Lin to identify a faulty instruction, the system asks a behavioral question: *What should the agent have done in this case?* Lin specifies that the itinerary should arrive before the presentation and records that consequential purchases should still require confirmation. Because the current workflow does not satisfy the first expectation, the row becomes a *change* commitment.

A single incident does not establish how broadly this change should apply. SYSTEMNAME therefore suggests a small number of contrastive cases, each accompanied by its source and the factor changed from the incident. Lin accepts a routine leisure itinerary in which no fixed commitment justifies paying substantially more; she marks the current cost-sensitive behavior as one to preserve. She also accepts a case in which timely arrival requires an extreme fare increase. Lin does not yet have a general policy for this trade-off, so she leaves the case unresolved rather than allowing the system to infer a rule. The three cases form *scope anchors*: concrete behavioral commitments that the patch generator may use. Lin can add historical executions or reject synthetic cases that she considers implausible.

Before generating a patch, the system freezes one or more additional *challenge cases*. Lin reviews each challenge case for realism, but its desired result and execution are not given to the patch generator. These cases do not provide an automatic ground truth; after seeing the candidate behavior, Lin will decide whether the change is consistent with her intent. The separation reduces the risk of evaluating an AI-authored patch only on the examples used to draft it.

4.1.2 Approving patch intent and reviewing behavioral change. Lin asks AI to draft a revision. The system does not immediately apply generated skill text. It first shows a concise *patch-intent preview*: "Add a narrow exception for fixed, time-critical commitments; preserve routine cost preference and purchase confirmation; do not silently resolve extreme-fare trade-offs." Lin must approve or edit this behavioral summary. She may then inspect the textual diff or edit the skill directly, but manual ownership of every word is not required for her to retain policy authority.

After approval, SYSTEMNAME creates one active candidate revision. The patch generator receives the original skill, the approved intent, and the scope anchors, but not the withheld challenge cases. The system reuses cached baseline executions when the skill version, case, tool snapshot, and model configuration are unchanged. It then executes the candidate under matched conditions. Repeated runs are summarized as behavioral outcomes and stability indicators; raw traces remain available in the row drawer.

The table prioritizes mismatches rather than requiring Lin to inspect every run. The candidate now arrives before the presentation and preserves the cheaper option on routine travel. However, in an expensive-purchase anchor, it books without confirmation. A challenge case also changes in a way Lin has not yet reviewed. The interface therefore reports one preservation regression and one new case requiring judgment, rather than a single aggregate score or a claim that the patch "passed three of four tests."

4.1.3 Investigating a mismatch and deciding reuse. Lin selects the confirmation regression and chooses *Investigate*. A drawer remains attached to that row and proposes one bounded question: whether the new priority clause is coupled to both timely arrival and confirmation behavior in this case. The proposed probe states what will change, what will be held fixed, how many executions may be required, and what interpretations the result can support. Lin approves a temporary what-if run that disables only the new clause while replaying the same itinerary and tool state.

The result restores confirmation but also removes the desired conference behavior. SYSTEMNAME reports this as behavioral coupling in the reviewed cases, not proof of an internal root cause. The AI proposes a narrower revision that allows time-critical commitments to override cost and airline preferences but not confirmation requirements. Lin approves the revised patch intent, and the table updates after new candidate executions. The motivating change now transfers, the confirmation commitment is preserved, and the extreme-fare case routes the decision to Lin instead of silently selecting a policy. Lin judges the challenge case, adds it as a future regression anchor, and releases the patch. The system stores the approved scope, skill diff, execution configuration, evidence, and release rationale with the new skill version.

4.2 The Behavioral Patch Table

The Behavioral Patch Table is both the repair specification and the review surface. It is deliberately relative to an existing workflow: users need not fully specify how the agent should behave everywhere, but must state which observed behaviors a patch is meant to change, preserve, or leave undecided in the cases they review.

Case-level behavioral commitments. Each case c contains task context, a replayable environment snapshot when available, and a cached baseline behavior $B_0(c)$ produced by the current skill. The user assigns one of three dispositions: CHANGE, PRESERVE, or UNRESOLVED. For a change commitment, the user describes a desired observable behavior $I(c)$, such as selecting an itinerary that arrives before a fixed commitment. For a preservation commitment, $I(c)$ refers to the valuable property of the baseline that should remain stable, such as requesting confirmation before a purchase.

Behavioral Patch: Travel Recovery			
✓ 2 aligned ✗ 1 regression ? 1 needs judgment ▲ 1 insufficient evidence			
Case and commitment	Before → after	Review	Action
Conference tomorrow <i>Change: arrive before presentation</i>	Late, cheap → on time	✓ Changed as intended	Inspect
Routine trip <i>Preserve: substantial cost savings</i>	Cheap later → cheap later	✓ Preserved	Inspect
Expensive purchase <i>Preserve: ask before purchase</i>	Confirmed → auto-booked	✗ Unexpected change	Investigate
Extreme fare <i>Unresolved: no automatic policy</i>	Cheap → costly early	? Entered unresolved region	Decide
Missing tool response <i>Not yet classified</i>	Mixed → mixed	▲ Evidence unstable	Rerun
Primary actions: Add a case Revise patch Release Abstain		On demand: Textual diff Raw runs Targeted probe	

Figure 2: The Behavioral Patch Table is SYSTEMNAME’s central representation. The system has three-way semantics—current behavior, intended relative change, and candidate behavior—but presents a compact before–after comparison and a decision-relevant review status. Mismatches are expanded by default; aligned rows and technical evidence remain inspectable on demand. This schematic should be replaced by the audited system screenshot in the final manuscript.

A. Scope the patch Incident: arrive after presentation Change: arrive beforehand Preserve: routine cost preference Unresolved: extreme fare trade-off <i>User judgment defines the relative scope.</i>	B. Review the candidate ✓ Target behavior changed ✓ Routine behavior preserved ✗ Confirmation regressed ? Challenge case changed <i>The table surfaces mismatches first.</i>	C. Investigate and revise Targeted what-if on mismatch Evidence: clause affects two commitments Narrower patch drafted and rerun Release / revise / gather / abstain <i>Technical detail appears only when needed.</i>
--	--	--

Figure 3: The walk-through follows one continuous representation and three user actions: scope a relative behavioral change, review the candidate, and investigate a mismatch. Targeted probes and textual details are progressive disclosures rather than separate workspaces.

An unresolved case intentionally has no target behavior; it marks a policy boundary that the patch must not silently settle.

Cases can refer to a final outcome, an observable decision, a tool call, an action order, a confirmation point, or a combination of these. The system encourages the smallest behavior that the user can judge reliably. A commitment remains linked to its originating incident, historical execution, or generated contrast, and to a short user rationale. Users may edit the commitment or add counterexamples as their understanding changes; each revision creates a new scope version rather than overwriting the criteria under which an earlier patch was reviewed.

Three-way semantics with a two-way presentation. For each case, the system executes a candidate skill to estimate $B_p(c)$ and compares the observed delta $B_p(c) - B_0(c)$ with the user-approved intended delta. The interface does not require the user to manually scan three full representations. Instead, it presents (1) the case and commitment, (2) a compact before–after behavior, and (3) a review status. This supports the following distinctions:

The system reports these states separately rather than compressing them into an aggregate score. A high target success rate cannot cancel a consequential preservation regression, and an unresolved

Table 1: Decision-relevant review states in the Behavioral Patch Table.

State	Meaning
Changed as intended	A change commitment is supported by the candidate evidence.
Intended change missing	The candidate does not reliably achieve a change commitment (underreach).
Preserved	A preservation commitment remains supported.
Unexpected change	A preservation commitment is violated or materially altered (overreach).
Unresolved region changed	The candidate changes a case for which the user has not chosen a policy.
New case needs judgment	A withheld challenge case reveals behavior outside the approved scope.
Insufficient evidence	Runs, evaluator judgments, or environment state do not support a stable comparison.

exposure cannot be counted as a failure or success before the user makes a policy judgment. Aligned rows collapse into a summary by

default; regressions, unresolved changes, challenge cases, and unstable evidence remain expanded. Users can inspect the raw paired executions and evaluator provenance for any row.

Scope anchors and challenge cases. Scope anchors are cases whose behavioral commitments the user approved before patch generation. They serve as a relative specification and are visible to the patch generator. Challenge cases are selected and frozen before the candidate is drafted but withheld from the generator. They can be retrieved from historical logs or generated by varying a behaviorally relevant factor near the current scope. The user validates their plausibility before execution, but judges the candidate behavior only after it is revealed. Challenge cases therefore support transfer review without pretending that the system possesses complete ground truth for an open-ended workflow. A challenge case that surfaces a meaningful new commitment can be promoted to an anchor in the next scope version.

Approving behavioral intent before textual implementation. The AI synthesizes the anchors into a short patch-intent preview that states the intended exception, the commitments that must remain stable, and any unresolved boundary the patch should defer. The user must approve this summary before the system generates or applies skill text. The textual diff remains inspectable and editable, and can be made mandatory for high-risk workflows, but the required user judgment is the behavioral policy rather than the exact wording. This distinction enables domain workflow owners to retain authority without assuming skill-engineering expertise.

4.3 Mismatch-Driven Evidence and Revision

SYSTEMNAME converts execution into review evidence but does not make all experimentation part of the default workflow. Baseline behavior is established when a case enters the table and is cached while the skill version, case, tool snapshot, model configuration, and evaluator remain unchanged. The candidate is then run under matched conditions on the anchors and challenge cases. The system begins with a small number of repetitions and allocates additional runs when outcomes disagree, the evaluator is uncertain, or the user marks a case consequential. The default view shows the resulting behavior distribution and a stability label rather than every run.

Behavior evaluation and provenance. A case may use a deterministic trace assertion, structured extraction followed by a rule, an LLM-based semantic evaluator, or explicit human judgment. For example, whether confirmation preceded booking can be checked from the action trace; whether an itinerary “reasonably respects airline preference” may require semantic judgment. Every status remains linked to the evaluator type, its definition, the paired outputs, and any disagreement. When no evaluator is sufficiently reliable, the row displays the executions for user judgment rather than producing an automatic pass/fail label. The system never uses hidden chain-of-thought as evidence; it operates on task context, observable traces, tool states, actions, and outcomes.

Targeted probes on mismatching rows. A user may investigate the motivating incident before patch drafting, or an underreach, overreach, or unstable row after a candidate is reviewed. If the incident is not reproducible or its tool state is suspect, the same

row-attached mechanism supports rerunning or replaying it and can end in abstention without generating a candidate. The system proposes at most a small bounded set of probes, each expressed as a question rather than a diagnosis. A probe can repeat an execution, perturb one context factor, replay a tool state, or temporarily modify a skill clause. The proposal identifies the changed factor, held-constant factors, anticipated execution cost, and interpretations that the evidence could support. After user approval, results appear as an evidence card attached to the row. Skill probes can reveal that a clause is behaviorally coupled to multiple commitments in the reviewed cases, but the interface avoids claiming that an intervention exposes a unique internal cause.

Revision and reuse decisions. After reviewing evidence, the user can revise the behavioral commitment, ask AI to draft a narrower patch, edit the skill directly, add a case, accept an explicit exception, or abstain from the skill edit. A revised patch repeats the intent-approval and behavioral-review loop. Release is available only as a user decision, not an automatic test result. If a patch changes an unresolved case, the interface asks the user to define a policy, route the boundary to runtime confirmation, record an explicit waiver, or continue investigating. When released, the approved anchors, challenge-case judgments, patch, paired executions, and rationale become regression artifacts attached to the new skill version.

4.4 System Architecture, Implementation, and Scope

The prototype is implemented as a web application with four logical services: a table-centered client, an orchestration service that manages scope and patch versions, a sandboxed execution harness, and an evidence store. **[DETAIL REQUIRED]** *frontend and backend frameworks, deployment environment, and repository details* A skill import contains the versioned natural-language instructions, tool manifest, and execution configuration. The harness normalizes each run into observable events, including model outputs, tool calls and returns, user-confirmation requests, actions, and final outcomes. Side-effecting tools are replayed from recorded responses or replaced by dry-run adapters during review.

LLM services support contrastive-case suggestion, patch-intent synthesis, candidate patch drafting, behavioral summarization, semantic evaluation where needed, and probe proposal. **[DETAIL REQUIRED]** *models, versions, temperatures, prompt-routing logic, and context limits* The system keeps these roles separate in the execution record so that a patch generator cannot inspect withheld challenge cases or their outcomes. Deterministic assertions are preferred when a commitment can be expressed over the trace; semantic evaluators expose their prompt and exemplars and can be overridden by the user. Appendix A provides the data model, execution protocol, mismatch rules, prompt templates, and full walkthrough materials.

SYSTEMNAME supports evidence-grounded review rather than formal verification. Its evidence is conditional on the selected cases, model and tool versions, environment snapshots, repetition budget, and evaluator quality. Targeted skill interventions estimate behavioral sensitivity under those conditions; they do not reveal hidden model reasoning or guarantee a unique root cause. The current

design assumes that a reusable skill can be versioned, relevant executions can be sandboxed or replayed, and the workflow owner can judge consequential behavior. Failures that cannot be reproduced or safely simulated may remain inconclusive, in which case abstention or escalation is an intended outcome.

Model Prompts. arXiv:2503.05070 [cs.SE] doi:10.48550/arXiv.2503.05070

References

- [1] Ian Arawjo, Chelse Swoopes, Priyan Vaithilingam, Martin Wattenberg, and Elena L. Glassman. 2024. ChainForge: A Visual Toolkit for Prompt Engineering and LLM Hypothesis Testing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery. doi:10.1145/3613904.3642016
- [2] Alberto Bacchelli and Christian Bird. 2013. Expectations, Outcomes, and Challenges of Modern Code Review. In *Proceedings of the 35th International Conference on Software Engineering*. IEEE, 712–721. doi:10.1109/ICSE.2013.6606617
- [3] Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, and Qing Wang. 2026. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. arXiv:2606.06324 [cs.SE] doi:10.48550/arXiv.2606.06324
- [4] Will Epperson, Gagan Bansal, Victor Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery. doi:10.1145/3706598.3713581
- [5] K. J. Kevin Feng, Kevin Pu, Matt Latzke, Tal August, Pao Siangliulue, Jonathan Bragg, Daniel S. Weld, Amy X. Zhang, and Joseph Chee Chang. 2026. Cocoa: Co-Planning and Co-Execution with AI Agents. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery. doi:10.1145/3772318.3791673
- [6] Yuru Feng, Yaoqi Chen, Beidi Zhao, Qianxi Zhang, Xinjiang Wang, Jianan Lu, Zhirui Wang, Shusen Xu, Zengzhong Li, and Qi Chen. 2026. SkillHEX: Improving Agent Skills via Hypothesis-Driven Autonomous Exploration and Exploitation. arXiv:2608.05628 [cs.AI] doi:10.48550/arXiv.2608.05628
- [7] Sunnie S. Y. Kim, Jennifer Wortman Vaughan, Q. Vera Liao, Tania Lombrozo, and Olga Russakovsky. 2025. Fostering Appropriate Reliance on Large Language Models: The Role of Explanations, Sources, and Inconsistencies. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery. doi:10.1145/3706598.3714020
- [8] Tae Soo Kim, Yoonjoo Lee, Jamin Shin, Young-Ho Kim, and Juho Kim. 2024. EvalLM: Interactive Evaluation of Large Language Model Prompts on User-Defined Criteria. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery. doi:10.1145/3613904.3642216
- [9] Andrew J. Ko, Robin Abraham, Laura Beckwith, Alan Blackwell, Margaret Burnett, Martin Erwig, Christopher Scaffidi, Joseph Lawrance, Henry Lieberman, Brad Myers, Mary Beth Rosson, Gregg Rothermel, Mary Shaw, and Susan Wiedenbeck. 2011. The State of the Art in End-User Software Engineering. *Comput. Surveys* 43, 3, Article 21 (2011), 44 pages. doi:10.1145/1922649.1922658
- [10] Derek Koh, Jinghui Mo, Benjamin H. Le, Jiening Zhan, Baofen Zheng, Kevin Bevis, Nathaniel C. Owen, Lauren Elizabeth Charney, Wenqiong Liu, and Jingwei Wu. 2026. Contrastive Reflection for Iterative Prompt Optimization. arXiv:2606.30840 [cs.IR] doi:10.48550/arXiv.2606.30840
- [11] Michelle S. Lam, Fred Hohman, Dominik Moritz, Jeffrey P. Bigham, Kenneth Holstein, and Mary Beth Kery. 2025. Policy Maps: Tools for Guiding the Unbounded Space of LLM Behaviors. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery. doi:10.1145/3746059.3747680
- [12] Minjae Lee and Minsuk Kahng. 2026. Data-Prompt Co-Evolution: Growing Test Sets to Refine LLM Behavior. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery. doi:10.1145/3772318.3791222
- [13] Yuxuan Liu, Zhaochen Su, Lingyun Xie, Yuhao Zhang, Qing Zong, Jiahe Guo, Zhongwei Xie, Yiyan Ji, Yauwai Yim, Hongyu Luo, Xiyu Ren, Chenyu Ruan, Haoran Li, and Yangqiu Song. 2026. SkillRevise: Improving LLM-Authoring Agent Skills via Trace-Conditioned Skill Revision. arXiv:2606.01139 [cs.AI] doi:10.48550/arXiv.2606.01139
- [14] Quentin Romero Lauro, Shreya Shankar, Sepanta Zeighami, and Aditya Parameswaran. 2026. RAG Without the Lag: Enabling “What-If” Analysis for Retrieval-Augmented Generation Pipelines. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery. doi:10.1145/3772318.3790874
- [15] Gregg Rothermel and Mary Jean Harrold. 1996. Analyzing Regression Test Selection Techniques. *IEEE Transactions on Software Engineering* 22, 8 (1996), 529–551. doi:10.1109/32.536955
- [16] Reshabh K. Sharma, Jonathan De Halleux, Shraddha Barke, Dan Grossman, and Benjamin Zorn. 2025. PromptPex: Automatic Test Generation for Language

A System Specification and Reproducibility Details

This appendix specifies the behavioral patch record, case lifecycle, execution and evaluation protocol, mismatch classification, targeted probes, and prompt templates. Exact implementation values marked with **[DETAIL REQUIRED]** ... must be replaced with audited values before submission.

A.1 Behavioral Patch Record

A behavioral patch is a versioned record

$$\mathcal{P} = \langle S_0, V, C_A, C_H, I_P, S_P, E, J, R \rangle,$$

where S_0 is the original skill version, V is the repair-scope version, C_A is the set of scope anchors, C_H is the frozen challenge set, I_P is the approved patch-intent preview, S_P is the candidate skill, E is the paired execution evidence, J contains user judgments on unresolved and challenge cases, and R is the final reuse decision and rationale.

A.1.1 Case record. Each case stores the fields in Table 2. A case can enter the system from the motivating incident, a historical execution, a user-authored example, or an AI-suggested contrast. Synthetic origin is always visible.

A.1.2 Scope versioning. A scope edit creates a new immutable version when the user changes a disposition, target behavior, rationale, evaluator, or membership in C_A . This prevents a patch from being retrospectively judged under criteria silently changed after its behavior was seen. The interface shows a concise diff, such as “v2 added routine-cost preservation” or “v3 changed extreme-fare behavior from unresolved to runtime confirmation.” Candidate revisions record the scope version under which they were generated.

A.2 Case Construction and Withholding Protocol

Case sources. The system prioritizes observed history when available. AI-suggested cases are generated by modifying one or two factors that could alter the repair boundary, including task urgency, cost difference, available tool information, user preference, action consequence, or requirement conflict. Each suggestion states (1) its relation to an existing case, (2) the changed factor, and (3) why the case may distinguish a broad repair from a scoped one. The user can edit or reject it before any execution.

Anchor construction. A case becomes an anchor only after the user reviews its context and specifies Change, Preserve, or Unresolved. The patch generator receives the context and approved behavioral commitment for Change and Preserve anchors. For Unresolved anchors, it receives a constraint not to silently define the boundary; permissible implementations include preserving current behavior, asking the user at runtime, or explicitly leaving the case outside the patch scope.

Challenge construction. Before patch generation, the system proposes a small challenge set using historical retrieval or contrastive generation. The user validates plausibility, after which the contexts are frozen and assigned a hash. The patch generator receives neither the contexts nor evaluator outcomes. The execution service

may access them only after the candidate skill is committed. If the user edits a challenge context after seeing candidate behavior, it becomes part of a new review round rather than remaining a withheld case.

Challenge interpretation. A challenge case is not an automatic test with presumed ground truth. Its initial state is “New case needs judgment.” After reviewing the before–after behavior, the user can (1) accept the candidate behavior as consistent with the existing scope, (2) promote the case to a Change or Preserve anchor, (3) leave it unresolved, or (4) reject the case as implausible. Only promoted cases become regression anchors for later versions.

A.3 Patch-Intent Preview and Candidate Generation

The system separates approval of behavioral policy from inspection of textual implementation. Candidate generation follows four steps:

- (1) **Synthesize intent.** Summarize the narrowest policy consistent with Change anchors, preservation commitments, and unresolved boundaries.
- (2) **User approval.** Require the user to accept or edit the behavioral summary. The system highlights any inferred priority or exception not directly grounded in a case.
- (3) **Generate one candidate.** Draft one active revision from S_0 , the approved intent, and C_A . Do not provide C_H to the generator.
- (4) **Commit candidate.** Save the exact skill text and hash before challenge execution. The user may inspect or edit the diff; edits create a new candidate hash.

The candidate prompt instructs the model to minimize behavioral scope rather than textual edit distance, preserve unspecified instructions, and avoid resolving Unresolved anchors unless the approved intent explicitly routes them to user confirmation. The system may generate alternatives only when the user requests them; the default workflow reviews one active candidate to avoid turning the interface into a patch-ranking dashboard.

A.4 Execution, Adaptive Repetition, and Baseline Caching

Matched execution. Original and candidate comparisons use the same task context, model family and version, generation configuration, tool interfaces, and environment snapshot whenever replay is possible. Side-effecting tools run in a sandbox or dry-run mode. Each execution stores a configuration hash. A cached baseline remains valid only while all fields included in the hash are unchanged.

Adaptive repetition. The prototype begins with k_0 runs for each new candidate–case pair and permits at most $k_{\max}$. Additional runs are requested when normalized outcomes disagree, the evaluator is uncertain, or the user marks the case high consequence. **[DETAIL REQUIRED]** Report the audited values for k_0 and $k_{\max}$ and the implemented stopping thresholds. Figure 4 provides the intended control flow rather than a statistical guarantee.

User-facing aggregation. The default row displays a canonical behavior summary and distribution, for example “selected an on-time option in 5/5 runs.” When outcomes differ, the interface lists the major alternatives and labels the row Insufficient evidence.

Table 2: Case record used by the Behavioral Patch Table.

Field	Type	Purpose
Case ID and source	Identifier; incident, historical, user-authored, or generated	Establishes provenance and prevents generated cases from appearing as observed history.
Task context	Structured fields plus natural-language description	Reconstructs the task and identifies the factor changed in a contrastive case.
Environment snapshot	Tool results, data versions, and side-effect policy	Enables matched replay or records why replay is unavailable.
Baseline configuration	Skill hash, model and tool versions, generation settings	Determines whether a cached original execution remains valid.
Baseline evidence	Runs, normalized events, behavior summary, and stability	Represents $B_0(c)$.
User disposition	Change, Preserve, Unresolved, or Not Yet Classified	Specifies the relative role of the case in the patch.
Behavioral commitment	Observable target or invariant plus rationale	Represents $I(c)$ for Change or Preserve cases.
Evaluator	Deterministic, structured, semantic, or human	Documents how behavior is judged and its provenance.
Generator visibility	Anchor-visible or challenge-withheld	Prevents challenge leakage into candidate drafting.
Candidate evidence	Candidate runs, normalized events, summary, and stability	Represents $B_p(c)$.
Review state	One state from Table 1	Supports mismatch-first review.
User judgment	Accepted behavior, new commitment, exception, or unresolved	Records decisions on challenge or boundary cases.

Adaptive behavioral review

```

runs ← execute(candidate, case,  $k_0$ )
summary ← normalize_and_evaluate(runs)
while needs_evidence(summary, consequence)
  and  $|runs| < k_{max}$ :
    runs ← runs ∪
      execute(candidate, case, batch)
    summary ← normalize_and_evaluate(runs)
return behavior_distribution, stability, provenance

```

Figure 4: Intended adaptive-execution control flow. The final implementation must document the disagreement metric, batch size, stopping threshold, and treatment of evaluator uncertainty.

Individual executions, traces, and evaluator outputs are available in chronological order. The system does not hide failures behind a majority outcome.

A.5 Behavior Representation and Evaluators

The execution normalizer maps raw traces to observable events such as information retrieval, candidate generation, decision, tool invocation, confirmation request, action, and final outcome. A case can use multiple evaluators, each aligned to one behavioral commitment.

Table 3: Evaluator hierarchy and default presentation.

Evaluator	Use and review
Deterministic trace assertion	Preferred for explicit events or ordering, such as confirmation preceding booking. Show the assertion and matched events.
Structured extraction plus rule	Extract a typed value, such as arrival time, then apply a transparent rule. Show extracted values.
Semantic evaluator	Use when the commitment is qualitative. Show prompt, exemplars, output, and disagreement across repeated judgments.
Human judgment	Use when no automatic evaluator is defensible or when the user overrides one. Preserve the paired outputs and rationale.

Semantic evaluators are proposed from the user’s commitment but are not silently treated as ground truth. Before relying on an evaluator, the user can review positive and negative exemplars. **[DE-TAIL REQUIRED]** *semantic-evaluator model, number of judgments, aggregation rule, and audit protocol* Evaluator changes invalidate affected cached review states.

A.6 Mismatch Classification

Let $Q(c, B)$ denote the evidence-backed judgment that behavior B satisfies the user-approved commitment in case c . The system

applies the following conceptual rules; the implementation must report the exact thresholds used for stochastic outcomes.

$$\begin{aligned}\text{CHANGEDAsINTENDED}(c) &\iff D(c) = \text{CHANGE} \wedge Q(c, B_p), \\ \text{UNDERREACH}(c) &\iff D(c) = \text{CHANGE} \wedge \neg Q(c, B_p), \\ \text{PRESERVED}(c) &\iff D(c) = \text{PRESERVE} \wedge Q(c, B_p), \\ \text{OVERREACH}(c) &\iff D(c) = \text{PRESERVE} \wedge \neg Q(c, B_p).\end{aligned}$$

For an Unresolved anchor, the system compares B_p with B_0 only to detect whether the patch entered the region; it does not assign success or failure. If a material difference is observed, the state is Unresolved region changed. Challenge cases begin as New case needs judgment. Any row becomes Insufficient evidence when the behavior distribution, environment state, or evaluator disagreement falls below the implemented evidence criterion. Insufficient evidence takes precedence over substantive pass/fail classification.

A.7 Targeted Probe Protocol

A targeted probe is available only from a mismatching or unstable row. The system first generates a probe proposal with the schema below; execution requires user approval.

The default planner returns a small set of proposals ranked by relevance to the selected mismatch and bounded by the remaining execution budget. **[DETAIL REQUIRED]** Report the audited maximum number of probe suggestions. A skill probe operates on an isolated temporary copy and never mutates the candidate under review. Probe results can motivate a revision but are not substituted for executing the final candidate itself.

A.8 Reuse Decisions and Provenance

The user can choose:

- **Release:** create a new skill version with the reviewed patch record;
- **Revise:** edit the intent, skill text, or both and begin a new candidate round;
- **Gather evidence:** add repetitions, cases, or a targeted probe;
- **Abstain:** retain the current skill and record that evidence did not justify a persistent edit; or
- **Escalate:** mark the incident as likely requiring tool, data, routing, or broader system investigation.

A release record includes skill hashes, scope and evaluator versions, anchor and challenge judgments, paired executions, waivers, and user rationale. The interface does not claim certification. It reports the reviewed scope and open limitations, for example “released with three anchors, two promoted challenge cases, and one unresolved boundary routed to runtime confirmation.”

B Illustrative Walk-through Materials

The following materials instantiate the scenario in Section 4.1. They are design examples and must be replaced by outputs produced by the audited prototype before publication.

B.1 Original Travel-Recovery Skill

- (1) Retrieve the traveler’s affected calendar commitments and identify their time and location constraints.

- (2) Search feasible replacement itineraries and remove options that cannot satisfy hard logistical constraints.
- (3) Prefer lower total cost when options are otherwise practical.
- (4) Prefer the same airline and fewer connections when practical.
- (5) Ask the traveler for confirmation before a purchase above the configured consequential-purchase threshold.
- (6) Present the selected option and rationale, then rebook only after any required confirmation.

The incident leaves the priority among commitments, cost, airline preference, and confirmation underspecified. This skill is illustrative; the final paper should report the exact version used in the implemented walkthrough and evaluation.

B.2 Scope Anchors and Challenge Case

B.3 Illustrative Patch Intent and Revisions

Approved intent preview. “Add a narrow exception for fixed, time-critical commitments. When necessary, arrival feasibility may override cost and airline preferences, but it must not override consequential-purchase confirmation. Where the fare trade-off remains unresolved, request the traveler’s decision rather than choosing automatically.”

Candidate v1. An illustrative first revision adds a broad priority clause for time-critical commitments. In the scenario, this candidate achieves the motivating case but is observed to bypass confirmation under an expensive purchase. This outcome must be produced by the actual prototype before being used as a factual walkthrough claim.

Candidate v2. The revised illustrative clause narrows which preferences can be overridden and explicitly preserves confirmation. The final implementation should report the exact textual diff and paired executions rather than the summary used here.

C LLM Prompt Templates

The templates below specify task contracts and output schemas. They should be populated with the implemented model names, examples, and safety checks. Prompts must not expose withheld challenge cases to the candidate-patch generator.

C.1 Contrastive Case Suggestion

SYSTEM: Suggest a small number of plausible cases that help a workflow owner clarify the behavioral boundary of a proposed repair. Do not decide the correct policy for the user.

INPUT:

- Motivating case and observable execution
- Current skill (for context, not diagnosis)
- Existing user-approved cases and commitments
- Available historical cases

FOR EACH SUGGESTION RETURN:

1. case_context
2. source: historical | generated
3. derived_from_case_id
4. changed_factors (one or two only)
5. why_this_case_may_clarify_scope
6. realism_assumptions

CONSTRAINTS:

Table 4: Targeted probe proposal and evidence-card schema.

Field	Proposal	Result card
Question	A discriminating behavioral question, not a root-cause claim.	Restated with the evidence scope.
Intervention	Repeat, context perturbation, tool-state replay, or temporary skill modification.	Exact executed change and configuration hash.
Held constant	Task factors, skill clauses, tool state, and model settings not changed.	Deviations or replay limitations.
Relevant commitments	Rows that the intervention may clarify.	Observed effects on each row.
Budget	Maximum executions and estimated cost.	Actual executions, latency, and cost.
Interpretations	What outcomes could support or weaken; what remains unresolved.	Supported, weakened, or inconclusive interpretations.
Evidence	Not applicable before approval.	Behavior distribution, stability, raw runs, and evaluator provenance.

Table 5: Illustrative cases used in the walk-through.

Case	Key context	Disposition	Behavioral commitment or review purpose
Conference incident	Fixed presentation the next morning; cheap option arrives afterward	Change	Select an itinerary that arrives before the presentation, subject to confirmation requirements.
Routine leisure trip	No fixed commitment; later option is substantially cheaper	Preserve	Continue to prefer the substantially cheaper practical option.
Expensive purchase	Replacement price exceeds consequential-purchase threshold	Preserve	Request confirmation before booking.
Extreme fare boundary	Timely option has a very large fare premium	Unresolved	Do not silently define an automatic cost-versus-commitment policy.
Withheld challenge	Time-critical commitment; two on-time options differ in cost and connection count	Challenge	Examine whether the patch generalizes as a narrow exception or over-prioritizes earliest arrival.

- Include at most [N] suggestions.
- Prefer historical cases.
- Do not label the desired behavior.
- Do not claim that a skill instruction caused the failure.

- Never silently resolve an Unresolved case.
- Phrase behavior in observable terms.

C.2 Patch-Intent Synthesis

SYSTEM: Produce a behavioral intent preview for a skill revision. Summarize what should change, what must remain stable, and what remains unresolved. Do not write skill text yet.

INPUT:

- Change anchors with desired observable behavior
- Preserve anchors with valued baseline behavior
- Unresolved anchors
- User rationales

RETURN:

1. intended_exception_or_policy
2. preserved_commitments
3. unresolved_boundaries_and_default_handling
4. inferred_priorities_requiring_user_approval
5. unsupported_assumptions

CONSTRAINTS:

- Use the narrowest policy supported by the cases.

C.3 Candidate Patch Drafting

SYSTEM: Revise the reusable skill to implement the user-approved behavioral intent. Minimize unsupported behavioral change, not merely textual change.

INPUT:

- Original skill
- Approved patch-intent preview
- Scope anchors and commitments
- Tool and format constraints

DO NOT RECEIVE:

- Withheld challenge cases
- Challenge executions or evaluator results

RETURN:

1. revised_skill
2. textual_diff_summary
3. mapping_from_each_edit_to_approved_intent
4. assumptions_and_known_limitations

CONSTRAINTS:

- Preserve instructions not implicated by the approved intent.

- Do not invent a policy for unresolved boundaries.
- Do not remove confirmation or safety constraints unless explicitly approved.

C.4 Behavioral Outcome Summarization

SYSTEM: Summarize observable workflow behavior across repeated executions. Use only task context, tool calls and returns, visible decisions, actions, and outcomes. Do not infer or report hidden chain-of-thought.

INPUT:

- Case context
- Normalized observable traces for repeated runs
- Behavioral commitment, if available

RETURN:

1. canonical_behavior_summary
2. outcome_categories_with_counts
3. observable_evidence_for_each_category
4. evaluator_result_and_provenance
5. stability_or_disagreement
6. information_missing_for_judgment

C.5 Targeted Probe Proposal

SYSTEM: Propose a bounded experiment that may clarify the selected behavioral mismatch. Frame the output as a question, not a root-cause diagnosis.

INPUT:

- Selected case and mismatch
- Original and candidate skill diffs
- Paired observable executions
- Relevant commitments
- Remaining execution budget

RETURN:

1. question
2. intervention_type
3. exact_change
4. held_constant
5. possible_outcomes_and_interpretations
6. commitments_that_may_be_affected
7. maximum_execution_budget
8. limitations

CONSTRAINTS:

- Change the fewest factors possible.
- Do not mutate the candidate under review.
- Do not claim causal identification beyond the executed conditions.

D Operational Safeguards and Audit Checklist

Before reporting or evaluating the system, the implementation should satisfy the following checklist:

- **Version pinning:** store skill, model, tool, prompt, evaluator, and environment versions for every run.
- **Challenge isolation:** verify through logs that candidate generation never receives withheld challenge content or outcomes.
- **Side-effect safety:** replay or dry-run consequential tools; do not purchase, send, delete, or modify external state during review.
- **Evidence traceability:** every row status links to the exact runs and evaluator outputs that support it.
- **Evaluator audit:** report deterministic coverage, semantic-judge validation, disagreement, and human overrides.

- **No hidden reasoning claims:** never present model chain-of-thought or model-generated explanations as execution evidence.
- **Failure visibility:** preserve minority outcomes and inconclusive probe results rather than summarizing only the modal behavior.
- **Scope provenance:** preserve when and why the user changed a commitment, added a case, or accepted an exception.
- **Abstention:** allow the workflow to end without a skill edit when evidence is weak or points outside the skill layer.
- **Walk-through reproducibility:** replace all illustrative outputs in Section 4.1 with runs from the frozen prototype and release the corresponding anonymized artifacts when possible.

E Participant Recruitment and Sample Construction

E.1 Sampling rationale

The study is designed to examine workflow ownership without assuming skill engineering expertise. We therefore recruit for high *workflow accountability* and variation in *artifact familiarity*. Workflow accountability means that the participant defines task goals, judges the acceptability of agent behavior, depends on the workflow's output, or bears responsibility for its consequences. Artifact familiarity concerns the extent to which the participant can inspect, author, or modify reusable instructions. We target 12–16 completed sessions, with a planned stopping point of 14 unless attrition or an imbalanced sample requires up to two additional participants. This target is intended to provide information-rich variation rather than statistical representativeness.

We seek variation along the following dimensions:

E.2 Inclusion and exclusion criteria

Participants must satisfy all inclusion criteria:

- (1) They have used an LLM assistant or agent for the same recurring family of tasks at least five times in the previous three months.
- (2) The workflow is shaped by a persistent instruction or configuration artifact, such as custom instructions, a saved system prompt, a reusable template, a routine, or an agent skill. The participant need not have authored it, but must be able to describe its role and provide at least a redacted excerpt, reconstruction, or screen-shared view.
- (3) They can identify an execution within the previous six months that they considered unacceptable, surprising, or unsafe to reuse without review.
- (4) They had responsibility for judging the workflow's goals or outcomes, even if another person or AI implemented the configuration.
- (5) They can provide a redacted failure artifact and either a typical successful execution or a description of expected behavior.

We exclude workflows whose discussion would require disclosure of protected health information, active legal representation, classified material, credentials, or other information that cannot be safely redacted. We also exclude one-off prompting, workflows without any persistent configuration, and participants who cannot describe a concrete breakdown. The study examines repair reasoning, not the correctness of decisions in regulated domains.

Table 6: Maximum-variation sampling dimensions. These are sampling targets, not experimental conditions.

Dimension	Lower end	Higher end
Artifact familiarity	Rarely inspects or edits reusable instructions; may ask a teammate or AI to make changes	Regularly authors, inherits, reviews, or versions system prompts, routines, or skills
Workflow consequence	Primarily time, quality, or inconvenience costs	Material financial, coordination, reputational, or operational consequences, excluding regulated high-stakes decisions
Workflow structure	Primarily generates or transforms information	Uses multiple tools, priorities, confirmations, or conditional actions
Ownership arrangement	Individual workflow	Shared, inherited, or organizational workflow
Domain	Research, writing, analysis, or personal productivity	Product, operations, software, customer support, or other organizational work

E.3 Recruitment screener

The screener should be administered before scheduling. Bracketed response options are suggestions and may be adapted to the recruitment platform.

- (1) Describe one task for which you repeatedly use the same AI assistant, agent, custom GPT, saved prompt, template, routine, or skill.
- (2) Approximately how many times have you used this workflow in the last three months? [1–2; 3–4; 5–10; 11–25; more than 25]
- (3) Who decides whether the workflow’s output or action is acceptable? [Primarily me; shared; primarily another person]
- (4) Who can inspect or change the reusable instructions? [Me; a teammate; AI on my behalf; platform owner; unknown]
- (5) Have you encountered a concrete execution that caused you to modify, consider modifying, reduce reliance on, or stop using this workflow? [Yes/No; brief description]
- (6) What could go wrong if the workflow behaves incorrectly? [Time loss; poor quality; financial cost; coordination cost; reputational cost; other]
- (7) Can you provide a redacted screenshot, output, trace, instruction excerpt, version diff, or reconstruction related to that incident? [Yes/No]
- (8) Does the material contain information that cannot be safely redacted or discussed in a research session? [Yes/No]

E.4 Participant table template

Populate Table 7 only after the sample is final. Use broad role and domain labels to avoid re-identification.

F Pre-Study Artifact Collection and Data Handling

F.1 Artifact request

Two to three days before the session, participants receive the following request:

Please choose one recurring task for which you use an AI assistant or agent with a persistent configuration, such as custom instructions, a saved prompt, template, routine, or skill. Provide, if available: (1) one execution that you considered unacceptable or surprising; (2) one typical or successful execution from the same workflow; (3) the relevant reusable instructions or a redacted excerpt; and (4) any note, version history, or change you considered after the incident. You may redact names, organizations, values, and proprietary details, reconstruct the case with equivalent placeholders, or show the materials only through screen sharing during the session.

Participants also answer a brief artifact inventory:

- (1) What recurring task was the workflow intended to support?
- (2) Who created and who currently maintains the reusable configuration?
- (3) Which parts of the configuration can you inspect or edit?
- (4) What happened in the selected breakdown?
- (5) What did you do afterward: rerun, edit, ask someone else to edit, manually take over, rollback, reduce automation, or stop using the workflow?
- (6) What behavior did you hope would remain unchanged?
- (7) What evidence, if any, convinced you that the incident was resolved?

F.2 Privacy and data minimization

Participants are instructed not to submit credentials, private keys, protected health information, privileged legal material, or data they are not authorized to share. The research team stores only the minimum artifact segments needed to understand the incident. Screen-shared materials not approved for retention are not captured in screenshots; the interviewer records a de-identified structural summary. Names, company identifiers, exact monetary values, and unique workflow details are generalized during transcription. The consent form separately asks permission to quote de-identified speech and to reproduce redacted artifact fragments. Every quotation used in the paper must be checked against the audio or transcript and approved under the study’s consent procedure.

G Complete Session Protocol

G.1 Pilot and freeze procedure

Conduct two pilot sessions that are excluded from the reported sample. The pilots test whether participants can understand the standardized scenario, whether the open evidence-request phase yields

Table 7: Participant characteristics. Replace all placeholders with verified, de-identified information.

ID	Role / domain	Recurring workflow	Artifact access	Artifact familiarity	Primary consequence
P1	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P2	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P3	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P4	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P5	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P6	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P7	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P8	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P9	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P10	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P11	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P12	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P13	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]
P14	[ROLE]	[WORKFLOW]	[ACCESS]	[LEVEL]	[CONSEQUENCE]

actionable requests, whether the common evidence fits within the allotted time, and whether the decision options are interpreted distinctly. After the pilots, revise ambiguous wording and then freeze: (1) the moderator script; (2) the standardized skill and incident; (3) the evidence bank; (4) the common evidence set; (5) response-card wording; and (6) the coding event schema. Any post-freeze change must be logged with its reason, date, affected participants, and analytic consequence.

G.2 Session overview

Table 8: Session structure and primary purpose.

Time	Activity	Purpose	Primary data
0–5 min	Consent and orientation	Establish scope; emphasize that the task has no single expected diagnosis	Consent; baseline description
5–30 min	Real break-down reconstruction	Examine current practice without imposing the triadic model	Timeline; artifact references; explanations; actions
30–65 min	Open-ended WoZ repair probe	Observe evidence seeking, scope revision, implementation choice, and action	Requests; predictions; proposals; final decision
65–80 min	Reflection and delegation interview	Compare probe and real practice; clarify desired authority	Interview; delegation sort; rationale

The nominal duration is 80 minutes. Small deviations are permitted only to let a participant finish a thought; the moderator must not selectively extend evidence seeking or decision time. The facilitator should not reward a particular diagnosis, encourage a skill edit, or imply that more evidence is always preferable.

G.3 Opening script: 0–5 minutes

The moderator reads:

We are studying how people understand and respond when a recurring AI workflow does something they consider unacceptable. We are evaluating the workflow and our study materials, not your skill. There may be more than one reasonable way to interpret an incident, and you do not need to identify one correct cause. During the second activity, you may request evidence from a prepared set. The assistant will not have unlimited capabilities, and an unavailable request is still useful data about what you wanted to know.

The moderator confirms consent, recording preferences, artifact-retention permissions, and the participant’s preferred terminology for the reusable configuration.

G.4 Part 1: Artifact-grounded breakdown reconstruction, 5–30 minutes

The moderator opens a neutral event timeline with six headings: *Goal*, *What happened*, *What I observed*, *What I inferred*, *What I considered changing*, and *What remained uncertain*. The headings are presented as note-taking aids, not as a model of the system.

A. Establish the recurring workflow.

- (1) What recurring work was the AI meant to perform?
- (2) What makes two uses part of the same workflow rather than unrelated prompts?
- (3) What outcomes, intermediate decisions, tool actions, or confirmation points matter to you?
- (4) Who is accountable when the workflow behaves incorrectly?

B. Reconstruct the incident chronologically.

- (1) What did you ask the system to do, and what context was available?
- (2) What did you directly observe in the output, trace, or tool results?
- (3) What do you believe happened but did not directly observe?
- (4) At what point did the workflow depart from your expectation?
- (5) What consequence made the outcome unacceptable?

The moderator asks the participant to mark each item as *observed*, *inferred*, or *uncertain*. The moderator does not correct the participant's account or introduce instruction-level terminology unless the participant does so.

C. Elicit possible objects of repair.

- (1) At the time, what did you think might need to change?
- (2) What alternative changes or non-changes did you consider?
- (3) What evidence supported each possibility?
- (4) Were there explanations you could not distinguish?
- (5) Was there any reason not to edit the reusable instructions?

Only after the participant has answered these open prompts may the moderator use a neutral rescue prompt: "Did you consider changing anything about the desired behavior, the persistent configuration, the current input, a tool, the amount of automation, or something else?" The transcript and coding record whether an object of repair was raised spontaneously or after this prompt.

D. Reconstruct the actual response and release judgment.

- (1) What did you or someone else actually change?
- (2) Did you test only the motivating case or other cases as well?
- (3) What behavior did you want to remain dependable?
- (4) What made you willing to keep using, deploy, roll back, or abandon the revised workflow?
- (5) Looking back, what information was missing?

At the end of Part 1, the moderator writes a neutral one-sentence summary and asks the participant to correct it. The summary must not identify a root cause unless the participant has done so.

G.5 Part 2: Open-ended Wizard-of-Oz repair probe, 30–65 minutes

G.5.1 Stage A: Incident only, 30–36 minutes. Show only the materials in Appendix H.1: the travel request, observable tool calls, candidate flights, agent decision, and missed presentation. Do not show the skill or an explicit list of requirements.

Ask:

- (1) What, if anything, is unacceptable about this execution?
 - (2) What should have happened in this case?
 - (3) What might you change first, if anything?
 - (4) What other future behavior could that change affect?
 - (5) How confident are you in your current explanation and action?
- Record each from 0 (not at all confident) to 100 (completely confident).

Do not use the labels *change*, *preserve*, or *unresolved* at this stage.

G.5.2 Stage B: Reveal the reusable artifact, 36–42 minutes. Reveal the proposed six-instruction skill in Appendix H.2 and the available tools. Give the participant time to read, then ask:

- (1) What does the skill change about your explanation, if anything?
- (2) Which parts of the skill seem relevant, and which relationships are you inferring rather than observing?
- (3) Does the skill fully express the policy you think the workflow should follow?
- (4) Would you change your proposed action?
- (5) Update explanation and action confidence from 0–100.

The participant then predicts the workflow's decision in the three short contexts in Appendix H.3 without seeing their outputs. These predictions elicit the participant's current mental model; they are not used to make inferential performance claims in the formative study.

G.5.3 Stage C: Free evidence request, 42–52 minutes. The moderator says:

You may now ask for any evidence that would help you decide what, if anything, should change. You can ask to see another type of case, rerun a case, inspect a tool result, try a what-if change, compare two revisions, or request something else. Tell us what you hope each request will help you decide.

For each request, the facilitator records:

- the participant's wording;
- the uncertainty or decision the request is intended to address;
- the matched evidence-bank card or an "unavailable" response;
- the participant's interpretation after seeing the evidence;
- any change in proposed object of repair, scope, confidence, or action.

The facilitator follows the lookup protocol in Appendix I. No new model output, explanation, or diagnosis may be generated during the session.

G.5.4 Stage D: Common evidence, 52–59 minutes. Present four evidence cards to every participant, using the counterbalanced orders in Table 11:

- (1) a routine preservation context;
- (2) a difficult policy-boundary context;
- (3) repeated executions of the motivating case;
- (4) a context in which calendar/tool state is incomplete.

If a participant already requested one of these cards, mark it as previously seen and spend no additional time on it. After each new card, ask:

- (1) What, if anything, does this change about your explanation?
- (2) Does it change what you would modify or where the modification should apply?
- (3) Does it reveal behavior that should remain stable?
- (4) Does it reveal a trade-off you are not ready to settle?

Only after these open questions does the moderator introduce a worksheet with three columns: *Should behave differently*, *Should remain stable*, and *Not ready to decide*. The participant may leave any column empty.

G.5.5 Stage E: Implementation and action choice, 59–65 minutes. Ask the participant to select one implementation mode:

- (1) write a skill revision manually;
- (2) ask AI to draft a revision and edit or approve it;

- (3) describe the desired behavioral policy and let AI implement the text;
- (4) defer implementation pending more evidence;
- (5) make no skill edit and address another layer.

If the participant requests an AI draft, provide a precomputed candidate that corresponds to their selected policy class, not a live personalized generation. The participant may annotate or reject it. Then show the available old-versus-new evidence cards and ask for one final action:

- **Release:** reuse the revision under the reviewed conditions;
- **Revise:** change the candidate before reuse;
- **Gather more evidence:** withhold a decision;
- **Abstain:** make no skill edit;
- **Escalate:** investigate a tool, context, platform, or owner outside the skill.

Record the participant's rationale and the evidence they consider decisive or missing. The moderator must not imply that release is the desired endpoint.

G.6 Part 3: Reflection and delegation interview, 65–80 minutes

Compare real and standardized incidents.

- (1) What felt similar or different between this scenario and your own workflow?
- (2) Did the probe cause you to reconsider how you handled your real incident?
- (3) Which evidence was useful, distracting, or missing?

Representation and legibility.

- (1) Would you rather begin from behavior, the skill text, or an execution trace? Why?
- (2) What should be shown by default, and what should require drill-down?
- (3) Do you need a responsible-instruction explanation, or is it enough to know how a candidate revision changes behavior?
- (4) How should unstable or inconclusive evidence be represented?

Division of labor. The participant sorts the responsibilities in Table 14 into *User*, *AI*, or *Shared*, then discusses any responsibility whose allocation depends on risk or artifact familiarity.

Release and maintenance.

- (1) What does “remaining in control” mean in this workflow?
- (2) What evidence would be required before release in your own domain?
- (3) What should be saved for the next maintainer or future revision?
- (4) When should the system recommend no skill edit?

G.7 Debrief

Explain that the standardized scenario was designed to admit multiple plausible repairs and that the study did not contain a hidden correct diagnosis. Clarify which outputs were precomputed, remind the participant not to adopt the study's candidate patches in a real workflow without independent validation, and confirm whether submitted materials may be retained.

H Standardized Travel-Recovery Scenario

Draft-integrity note. The materials in this appendix are a proposed study instrument based on the paper's motivating travel-recovery scenario. They are not copied from a validated dataset. Pilot, audit, and freeze all wording and executions before data collection.

H.1 Motivating incident

A researcher's original flight is canceled at 14:00 on the day before a conference presentation. The agent has access to the traveler's itinerary, calendar, airline search, fare details, and rebooking tool. The observable calendar result contains a paper presentation at 09:00 the following morning. The search tool returns the two feasible options in Table 9.

The agent selects Option A and explains that it is cheaper, uses the original airline, and avoids a connection. It does not ask for confirmation. The study shows the participant only observable inputs, tool outputs, and the final decision; it does not reveal hidden chain-of-thought or claim that one internal reason caused the decision.

H.2 Proposed reusable skill

The skill is intentionally plausible but incomplete: it asks the agent to retrieve relevant information and balance several preferences without fully specifying how time-critical commitments override them.

- (1) Identify all disrupted flight segments and retrieve feasible alternatives for the traveler.
- (2) Check the traveler's calendar for commitments within 24 hours of the original arrival time and include them in the comparison.
- (3) Prefer the original airline when a feasible option is available.
- (4) Prefer lower total fare and fewer connections among feasible options.
- (5) Ask for confirmation before selecting an option priced above \$500 or before a non-refundable purchase.
- (6) When no option satisfies all preferences, choose the option that minimizes overall travel disruption and explain the trade-off before rebooking.

The six instructions create several plausible objects of repair without making one a study-defined ground truth: the calendar information may be insufficiently prioritized; airline or cost preferences may be overly broad; “overall travel disruption” may be underspecified; confirmation may be too narrowly triggered; or a tool/context failure may make a textual edit inappropriate.

H.3 Prediction items after skill reveal

Before any additional evidence, ask participants to predict the original skill's likely choice and provide 0–100 confidence for each context:

- (1) **Routine leisure trip.** No calendar commitment depends on early arrival. A \$120 same-airline nonstop arrives at 15:00 next day; a \$840 different-airline one-stop arrives at 21:00 the same day.
- (2) **Moderate commitment, aligned preferences.** A \$260 same-airline nonstop arrives at 20:00 before a 10:00 meeting; a \$90 same-airline flight arrives at 11:30 after the meeting begins.

Table 9: Candidate flights in the motivating incident.

Option	Arrival	Airline	Fare	Connections	Consequence
A	13:30 next day	Original airline	\$80	0	Misses the 09:00 presentation
B	20:00 same day	Different airline	\$420	1	Arrives the evening before the presentation

(3) **High-cost boundary.** A \$1,180 different-airline flight arrives before a fixed 08:00 client presentation; a \$95 same-airline flight arrives six hours after it begins.

The formative study uses these predictions to elicit participants' current interpretation, not to estimate population-level prediction accuracy.

H.4 Common evidence cards

E1: Routine preservation context. Use the routine leisure trip above. The original skill's precomputed executions should predominantly favor the lower-cost same-airline option. The evidence card should expose raw decisions and explanations across repetitions, without labeling this behavior as correct.

E2: Policy-boundary context. Use the high-cost boundary above. Precomputed outputs should illustrate the trade-off between a fixed commitment and a large fare difference. It is acceptable for outputs to vary, provided that the card accurately reports the variation and does not manufacture a clean effect.

E3: Repeated motivating executions. Show five executions of the motivating incident under identical visible inputs. Report each option selected, whether confirmation was requested, and the observable justification. Do not average away disagreement.

E4: Incomplete calendar/tool state. Show a separate execution in which the calendar tool returns an incomplete result because the event is stored in a secondary calendar that is not queried. The purpose is not to claim that the motivating failure was a tool failure, but to ensure that participants can consider a non-skill object of repair.

H.5 Candidate implementation classes

Candidate text is shown only when requested in Stage E. The study should include at least these classes, each clearly labeled as a candidate rather than a recommended fix:

P1: Broad arrival-priority patch. "Always select the option with the earliest arrival, even when it costs more or uses a different airline." This patch fixes the motivating case but is intentionally likely to overreach in routine travel.

P2: Scoped commitment patch. "When the calendar contains a fixed commitment within 16 hours of arrival, select an option scheduled to arrive at least three hours before that commitment. Otherwise, continue to balance fare, airline, and connections. Ask for confirmation whenever satisfying the commitment requires a fare more than \$500 above the lowest feasible option." This patch encodes one possible boundary and confirmation policy; the study does not treat its thresholds as normatively correct.

P3: Uncertainty-and-confirmation patch. "If calendar information is missing, conflicting, or does not establish whether arrival time is consequential, present the trade-off and ask the user before re-booking." This patch addresses uncertainty rather than imposing a priority.

P4: No skill edit. Retain the skill and change calendar coverage, tool validation, or escalation outside the skill. This option ensures that the probe does not frame every breakdown as a textual repair problem.

I Wizard-of-Oz Evidence Bank and Facilitation Rules

I.1 Evidence-bank construction

Freeze the evidence bank before the first analyzed session. Use *[MODEL AND VERSION]* accessed or frozen on *[ACCESS/FREEZE DATE]*, with the same system configuration, tool outputs, sampling parameters, and execution harness for all participants. Record model identifier, endpoint, temperature, maximum tokens, tool schemas, prompt templates, and any unavailable seed control. Preserve raw inputs, tool events, outputs, timestamps, and evaluator annotations.

At minimum, precompute:

- five repeated executions for the original skill in each scenario context;
- five executions for each candidate patch in the motivating, preservation, and boundary contexts;
- selected rule-level what-ifs, such as omitting the airline preference, fare preference, calendar check, or a relevant pair;
- complete and incomplete calendar-tool states;
- old-versus-new comparisons that expose both local success and possible behavioral overreach.

These runs support the formative probe; they do not substitute for the later technical validation of any intervention method. If the precomputed model does not naturally produce the intended variation, report the actual evidence and revise the study materials during the pilot rather than manually editing model outputs.

I.2 Evidence-card format

Every evidence card contains:

- (1) the participant's request or the common question addressed;
- (2) the exact context and configuration compared;
- (3) a compact summary of decisions and observable actions;
- (4) the number of repeated executions;
- (5) a visible indicator of agreement or variation;
- (6) expandable raw outputs and tool events;

- (7) a neutral limitation statement, such as “This comparison does not establish a unique cause.”

The card must not contain a generated root-cause explanation, a responsible- instruction label, or language that endorses a candidate patch.

I.3 Request taxonomy and lookup

The taxonomy in Table 10 is used only by the facilitator to retrieve a card; it is not shown to participants.

Table 10: Wizard request taxonomy.

Request type	Examples	Response rule
Context request	“Show a case without a hard commitment”	Return nearest precomputed context; state differences explicitly
Repeat request	“Run the same case again”	Return all remaining repetitions, not a selected favorable subset
Artifact what-if	“Ignore the airline rule”	Return corresponding fixed intervention card
Tool-state request	“Did the calendar return the event?”	Return exact observable tool result; do not infer hidden state
Patch comparison	“Compare broad versus scoped edits”	Return fixed side-by-side old/new evidence
Raw evidence	“Show me the underlying outputs”	Expand unabridged executions and tool calls
Unsupported request	Request has no frozen card	State that the evidence is unavailable; ask what decision it would inform; log the gap

When multiple cards could match, use a prewritten priority order based on semantic proximity, not on which card best supports the paper’s hypothesis. A second researcher should audit a random sample of session-to-card matches.

I.4 Common-evidence counterbalancing

Use the four orders in Table 11 in rotation. If the final sample is not divisible by four, assign the remaining participants to the least used orders.

I.5 Facilitator neutrality rules

The facilitator must:

- use the participant’s terminology rather than substituting “skill,” “policy,” or “workflow” prematurely;

Table 11: Counterbalanced order of common evidence cards.

Order	1st	2nd	3rd	4th
A	E1	E2	E3	E4
B	E2	E3	E4	E1
C	E3	E4	E1	E2
D	E4	E1	E2	E3

- ask what a request is intended to decide before returning evidence;
- return all relevant repeated runs rather than selecting clean examples;
- avoid praising additional testing or portraying release as success;
- avoid stating that an intervention demonstrates causality;
- record unavailable requests rather than approximating them with a live model;
- disclose when a candidate patch was precomputed and not personalized;
- keep a deviation log for any departure from the script.

J Study Instruments

J.1 Breakdown timeline worksheet

Table 12: Neutral breakdown-reconstruction worksheet.

Prompt	Participant notes	Evidence status
Goal / expected outcome	[SESSION NOTES]	Observed / inferred / uncertain
What happened	[SESSION NOTES]	Observed / inferred / uncertain
Possible explanations	[SESSION NOTES]	Evidence for / against / missing
Possible objects of repair	[SESSION NOTES]	Policy / artifact / enactment / external / other
Actions considered or taken	[SESSION NOTES]	Implemented / rejected / deferred
What remained uncertain	[SESSION NOTES]	Open question

The “possible objects of repair” categories are completed by the researcher after the interview, not shown to the participant during reconstruction.

J.2 Stage-transition measures

At the end of Stages A, B, C, and D, record:

- (1) current explanation or set of explanations;
- (2) proposed action or non-action;
- (3) confidence in the explanation (0–100);

- (4) confidence in the proposed action's scope (0–100);
- (5) behavior believed likely to change elsewhere;
- (6) behavior believed important to retain;
- (7) unresolved trade-offs;
- (8) evidence still desired.

These ratings support process tracing and within-participant comparison. Do not use them to claim causal effects of individual evidence cards or population-level calibration.

J.3 Repair-scope worksheet

After the common evidence set, use Table 13. Ask the participant to phrase entries behaviorally, not as instruction numbers.

Table 13: Repair-scope worksheet introduced only after open evidence seeking.

Should behave differently	Should remain stable	Not ready to decide
[PARTICIPANT ENTRY]	[PARTICIPANT ENTRY]	[PARTICIPANT ENTRY]

For each entry, ask which concrete context supports it and whether the judgment reflects a domain policy, an assumption about the current skill, or an observed execution.

J.4 Delegation cards

J.5 Final action record

For the selected action, record:

- implementation mode;
- final action category;
- evidence considered sufficient;
- evidence considered missing;
- accepted unresolved regions;
- perceived consequences of a wrong decision;
- who should be notified or involved in a real deployment;
- what should be preserved as a future regression case.

K Data Analysis and Claim Audit

K.1 Analytic units and provenance

Segment the corpus into *events*: a participant states an expectation, reports an observation, offers an explanation, requests or receives evidence, changes an interpretation, revises repair scope, delegates a responsibility, or makes an action decision. Every event retains links to participant, activity (real incident or probe), timestamp, artifact/evidence card, and preceding state. This provenance prevents quotations from being separated from the evidence conditions under which they occurred.

A main-text finding should not rest solely on stated preferences. Unless the finding is explicitly about current practice, support it with at least two data forms, such as a real-incident account plus a probe action, or a participant request plus a later interview rationale.

Table 14: Responsibilities sorted into User, AI, or Shared during the exit interview.

Responsibility	Follow-up prompt
Decide whether the observed behavior is acceptable	What knowledge or accountability is required?
Generate contrasting future contexts	When should AI ask before generating cases?
Repeat executions and summarize variation	How much raw evidence is needed?
Choose a skill-level what-if intervention	Should the user approve each intervention?
Judge whether a tool or external layer may be responsible	What evidence would support escalation?
Draft a textual patch	Must the user understand every textual change?
Edit or approve the final skill text	Does this depend on artifact familiarity or stakes?
Judge whether the candidate changed the intended behavior	What context coverage is enough?
Decide whether established behavior was preserved	Who defines preservation commitments?
Release, revise, gather evidence, abstain, or escalate	Who holds final authority and why?
Save rationale and regression cases for future maintenance	What should become persistent documentation?

K.2 First-cycle inductive event coding

The first cycle does not begin with the user–skill–workflow triad. Analysts code participants' own terms using the families in Table 15. The listed codes are starting prompts, not a closed codebook.

Analysts write a memo after each session identifying emergent codes, surprising requests, and evidence that contradicts the current story. The research team reviews these memos before modifying the codebook.

K.3 FQ1: Abductive cross-layer synthesis

After first-cycle coding, examine what participants treated as possible objects of repair and when they moved among them. Candidate second-cycle categories may include:

- intended policy or domain judgment;
- reusable instruction/configuration artifact;
- context-specific enacted behavior;
- tool, retrieval, orchestration, or platform condition;
- organizational ownership, permissions, or maintenance process;
- no repair / accepted variability.

The triadic account is retained only if intended behavior, reusable artifact, and enacted workflow form a recurring and explanatory

Table 15: Initial event-code families.

Family	Example codes	Analytic question
Expected behavior	goal, constraint, priority, exception, confirmation	What did the participant believe should happen?
Observed breakdown	output symptom, tool action, omitted step, consequence	What was directly observable?
Inference	presumed priority, hidden interpretation, guessed tool state	What was inferred rather than observed?
Candidate explanation	policy gap, instruction, interaction, variability, context, tool, other	Where might the mismatch reside?
Evidence	artifact text, raw run, repetition, contrastive case, what-if, version history	What supported or weakened an explanation?
Uncertainty	competing explanation, unstable behavior, unresolved policy, missing evidence	What remained unsettled?
Repair action	textual edit, context change, tool fix, manual takeover, rollback, no action	What change was considered or taken?
Scope move	narrow, broaden, reframe, preserve, mark unresolved, abandon	How did the intended reach change?
Delegation	user, AI, shared; evidence, implementation, policy, release	Who performed or owned the work?
Decision	release, revise, gather evidence, abstain, escalate	What action followed, and why?

structure across cases. If external or organizational objects are equally central, the model and paper claim must preserve them rather than forcing them into the triad. Compare participants with different artifact familiarity to determine whether a pattern reflects implementation expertise or a broader property of mediated workflow control.

K.4 FQ2: Evidence-to-action process tracing

Construct a trajectory table for every participant:

Distinguish three effects:

- (1) *confirmatory*: evidence changes confidence but not interpretation or scope;
- (2) *specification-changing*: evidence adds, removes, or changes a behavioral commitment or policy boundary;
- (3) *localization-changing*: evidence changes the presumed object of repair or leads to abstention/escalation.

Compare self-requested and common evidence. A claim that contrastive cases help construct repair scope requires evidence of specification-changing transitions, not merely higher confidence after a researcher-provided example.

K.5 FQ3: Enacted and stated delegation

Create a matrix with rows for evidence work, implementation work, policy judgment, and release decision, and columns for:

- what the participant requested AI to do in the probe;
- what implementation mode they selected;
- how they allocated the corresponding delegation card;
- what they said they would prefer in their own workflow;

- conditions that changed the allocation, such as consequence, familiarity, traceability, or uncertainty.

Treat disagreement among these forms as data rather than resolving it in favor of interview statements. For example, a participant may claim to prefer manual control but choose AI drafting when facing time pressure, or may delegate text editing while retaining a release veto.

K.6 Researcher process and reflexivity

Two researchers independently analyze an initial subset selected to maximize variation in artifact familiarity, workflow consequence, and ownership arrangement. They compare event boundaries, identify missing code families, and produce a shared event schema. One researcher then leads full-corpus coding; the second reviews at least 30% of sessions, all candidate negative cases, and all excerpts considered for the paper. Disagreement is used to refine theme boundaries rather than converted into a reliability statistic inconsistent with interpretive analysis.

Maintain:

- a decision log linking codebook and theme revisions to evidence;
- a deviation log for study-protocol changes;
- a quotation audit with source timestamp and consent status;
- a finding matrix showing supporting and disconfirming cases;
- separate memos for real incidents and standardized probe behavior before cross-comparison;
- a reflexivity memo describing the team's assumptions about skills, repair, user control, and the proposed system.

Table 16: Within-participant process-trace schema.

Time	Evidence requested / received	Interpretation before → after	Object of repair before → after	Scope move	Action / confidence move
[TIME]	[EVENT]	[CHANGE]	[CHANGE]	[MOVE]	[MOVE]

K.7 Falsification and claim thresholds

Before finalizing findings, explicitly test the propositions in Table 17. The purpose is not statistical falsification but protection against a study that can only confirm the intended system story.

Table 17: Evidence that would strengthen, weaken, or redirect the paper's core claims.

Proposition	Evidence that supports it	Evidence that weakens or redirects it
Breakdowns require cross-layer localization	Participants naturally move among policy, artifact, enactment, and external explanations; experts also report nonlocal uncertainty	Most participants reliably identify a single faulty instruction and predict behavior from text alone
Repair scope is constructed across cases	Evidence produces new preservation commitments, changed policy boundaries, unresolved regions, or no-edit decisions	Participants begin with complete policies; additional cases only confirm them
Control centers on policy and release, not necessarily manual text authorship	Participants delegate evidence or patch drafting while retaining behavior and release decisions	Participants either insist on complete manual textual ownership or willingly delegate policy and release without review
Behavioral patch review is needed beyond ordinary test-and-diff	Participants need to revise the object or specification of repair, not only compare fixed versions	The main difficulty is simply insufficient regression coverage after a known patch

A main finding should satisfy four thresholds:

- (1) it is supported by multiple participants and more than one data form;
- (2) its boundary or heterogeneity is reported;
- (3) at least one plausible disconfirming case has been sought and addressed;

- (4) the wording does not imply causal effectiveness that only a later controlled evaluation can establish.

If the evidence does not support the provisional finding titles in Section 3.2, replace them. In particular, use the controlled study—not the formative interviews—to claim improved workflow legibility, calibrated prediction, reduced repair-scope mismatch, or better release decisions.

K.8 Descriptive reporting

Permissible descriptive summaries include:

- number of incidents and candidate objects of repair;
- frequencies of spontaneously requested evidence types;
- numbers of narrow, broaden, reframe, defer, abandon, and no-edit transitions;
- implementation and final-action choices;
- counts of participants contributing to a theme.

Report denominators, distinguish real-incident and probe counts, and avoid phrases such as “most users” without the sample denominator. Confidence ratings and behavioral predictions are elicitation aids; unless the study is redesigned and powered accordingly, do not test them inferentially.

L Material Freeze and Reproducibility Checklist

Before the first analyzed session, archive the following with immutable version identifiers:

- (1) recruitment screener, consent form, and artifact request;
- (2) moderator script and participant worksheets;
- (3) standardized incident, skill, contexts, and candidate patch set;
- (4) all raw evidence-bank inputs and outputs;
- (5) model, parameter, tool, and harness configuration;
- (6) evidence cards and request-to-card lookup table;
- (7) common-evidence counterbalancing schedule;
- (8) pilot changes and the final freeze decision;
- (9) initial event schema, analytic memos, and deviation-log templates.

After data collection, archive:

- (1) de-identified transcripts and retained artifact fragments, subject to consent;
- (2) logs linking sessions to evidence cards and counterbalancing assignments;
- (3) coded event data and process traces;
- (4) codebook and theme revision history;
- (5) finding-by-participant and negative-case matrices;
- (6) quotation audit and consent status;
- (7) the exact counts and excerpts used in the manuscript.

If public release of participant data is not possible, release the standardized materials, evidence bank, moderator script, codebook,

synthetic examples of the analysis tables, and a data-access statement explaining restrictions.

Draft-integrity note. Before submission, remove all draft notes and red placeholders; replace the provisional process figure caption; report the actual sample, ethics status, model version, pilot changes, deviations, and negative cases; and ensure that every finding is traceable to audited data.